

Finding the First 100 Teams for Atlan Agent Registry — The Complete Working Document

Aryaman Singh · GTM candidate work-sample · finalised 2026-09-27. This is the long-form document behind the presentation: the thesis, every major decision with its reasoning and rejected alternatives, the corrections that reshaped the work, the landscape it has to win in, how distribution works outside and inside an organisation, a worked example on a real company, what was built and what is blocked, and a devil's-advocate pass on all of it.

Independent candidate exercise. Not affiliated with or endorsed by Atlan. skillsdrift is my own open-source tool; every public artifact carries this disclosure. Figures from third parties are cited in the Sources section; figures from my own scans are labelled as such.

The answer on one page

The pain. Inside a single AI tool, skill governance is increasingly solved: Claude Enterprise now has org skills with review, version history and per-skill usage; ChatGPT has workspace skills with owners. The pain lives at the **seams**: the same skill across Claude, ChatGPT/Codex, Copilot and Cursor; skills copied out of Gong or Salesforce into assistants; skills that encode a metric or a price that later changes; usage that is reported per vendor; and the custom-GPT retirement on 11 Dec 2026 forcing every workspace to repackage.

The ICP. Organisations running two or more AI platforms across two or more functions, where someone already owns “canonical process” (an AI Transformation Owner, DevEx or GTM Enablement). **Wedge:** the engineer who inherited the drift. **Pain feeler:** the owner of canonical process. **Buyer:** the Head of AI / Director of Enterprise AI — and the CDO in existing Atlan accounts. **Not yet:** single-AI-tool orgs; CX teams on Decagon or Sierra.

The channel. Open-source, GitHub-native distribution (CLI + a real data-drop on HN, Twitter and LinkedIn) for breadth. Inside each org, **hubs** for depth: teams whose knowledge hundreds of colleagues consume.

Inside the org, before any admin install. Local scan → org scan with the engineer's own GitHub access → duplicate-cluster PRs and “claim your skills” into other teams' repos → cross-functional skills shared as **links to the skill's page in the org's Atlan dashboard** over Slack, with permission (the skillsdrift-bridge skill) → a champion emerges and installs the GitHub App (confirmation, not entry) → the operating-model owner buys.

Activation. Skills in the registry with named owners and versions; ~10 people across ≥3 functions and ≥2 AI tools running governed skills; usage observable at whatever tier each tool exposes.

The math. If several teams inside one org count toward 100, the goal needs tens of organisations, not ~667. This is the first question for Atlan.

Status. Built: CLI, scanner service and index, cards, draft-gated X bot, GitHub App service, the skillsdrift-bridge skill. Blocked: drift.aryaman.tech resolves but does not serve yet; npm publish, App registration, X API access and the waitlist are open. There is no demand data yet — that is the most important gap.

Part I — Absorbing the product

The brief describes Atlan Registry as “*a governed home for the context behind agents*” — skills, instructions, tools, knowledge, permissions and environments, given stable identity and connected to owners, versions, access, dependencies, evals, usage and traces. The initial wedge is skills. The sentence that anchored everything: “*Two agents running the same model can behave like different systems because their skills, instructions, tools, knowledge, permissions, and environments differ.*”

On first attempt none of the three videos were reachable through any automated route and the strategy memo returned 401, so the first round was built from the challenge PDF, the speaker's written recap and webinar corroboration, with every lockout documented. In hindsight the right move in a first week was simply to ask Atlan for access; I note it here because the brief explicitly invites that. When the transcripts were recovered, they corrected my product understanding in load-bearing ways.

The demo shows a desktop app that integrates with harnesses and, on first run, scans the skills and sessions already on the laptop — no cold start. Personal, company and department workspaces with access-scoped visibility. A reporting surface for a named persona — the *AI transformation leader* who needs to know “whether

that investment is working” — showing spend, runs, active agents, cost per run and how often sessions use governed skills. Agent 360 with dependencies (“so changes do not happen in the dark”). Versioning with pinning. Discovery with compare-and-consolidate for duplicates. Plugin distribution scoped by workspace membership. An MCP surface. An announced security scan. And a Software Factory demo applying the model to CI, where pattern-mined improvements become proposed skill updates, “never an automatic rule change”.

The co-founder's talk supplies the thesis — performance = intelligence × context; only 1 in 5 AI use cases reach production; “context is IP” — and Atlan's own history: an agent-per-job era that failed on context cost, islands, traceability and drift, then a shared-context era (300 skills, 40 agents, built by the best SEO person and the best competitive-intel person) that hit dependency breakage, unclear skill ownership, hardcoded secrets and public-repo downloads.

Two conclusions shaped everything. First, the market is Atlan's own biography, scaled up. Second, the product is observability + governance + distribution, not a directory — and its most distinctive surfaces (cross-harness reporting, Agent 360, governed distribution) are team-and-org primitives. Atlan's own site now lists “Agent Skills — reusable, versioned, testable units of procedural knowledge” inside its Context Engineering Studio, so the registry extends something the company already sells.

Part II — The customer, and how the answer changed

The first answer

The first ICP was “the accidental owner”: a senior engineer at a 10–200 person company who introduced Claude Code or Codex skills informally and now owns the drift mess. The evidence was real — the sx HN thread (“a team that had 5 copies that had all drifted”), Dropbox-symlink workarounds, Anthropic's own docs once conceding “No versioning... No central registry”. It got the wedge person right and the org shape — and therefore the buyer — wrong. Six corrections followed; the first four reshaped the ICP, the last two came from this final pass.

Correction 1 — Enterprises

The demo's narrator is an AI transformation leader running department workspaces. Category proof points are enterprise-shaped (AWS's Agent Registry launched with Southwest, PepsiCo and Syngenta; Atlan's customers include Mastercard and General Motors), and enterprises have triggers that cannot exist at startup scale: multi-harness drift, key-person risk, supply-chain exposure. Security review stops being an exclusion and becomes a design requirement: the free artifact is local-only, read-only, zero-network, zero-telemetry and MIT.

Reconciling this with the brief matters, because Atlan wants *“dramatic distribution before we optimize for a traditional enterprise sales motion.”* Enterprise is the destination, not the entry. Entry stays bottom-up and self-serve; the enterprise buying center is where adoption ends up, and nothing before the buying moment requires a salesperson.

Correction 2 — The ICP is the buying center, not the person

At enterprise the person who downloads is never the person who pays. The buying center has four seats: the engineer (entry), the champion (tech lead / DevEx, or RevOps / enablement on the business side), exactly one decision-maker, and the blockers (AppSec, IT, procurement), treated as design requirements. Artifacts travel; people don't. Hop 1 (engineer → champion) is carried by drift evidence in lead language (“used by 4 teammates, 3 versions”). Hop 2 (champion → decision-maker) is carried by the scorecard and exec memo, timed by real or recurring triggers, and helped by a “pincer”: the public data-drop makes leaders ask “do we have this?” while the champion already has the answer.

Correction 3 — Activation is deployment, not signup

An activated team has (1) its skills in the registry with owners and versions, (2) ~10 people running governed skills, and (3) governed usage observable. Test: if they stopped tomorrow, would real work break? Adoption is pulled by value, not pushed by governance — in the demo people adopt because a skill is visibly better than DIY and colleagues already use it — so the ignition move is making each workspace's top 3–5 skills great. Observability is tiered: session telemetry for Claude Code and Codex, usage and share events elsewhere.

Correction 4 — Non-tech enters the fold

Atlan's dogfooding started in customer experience; the 300-skill brain was built by marketing; the demo walkthrough is a knowledge-worker flow. Engineering stays the cold-start wedge (GitHub does not reach

marketers); non-tech is the in-account depth. The activation bar is explicitly multi-functional — roughly 5 engineers, 3 GTM, 2 ops across multiple harnesses.

Correction 5 — Single-vendor governance is solved; the pain lives at the seams

A landscape sweep changed the pain statement. Since July 2026 Claude Team/Enterprise admins can publish org skills with a review workflow, version history, automatic propagation of approved versions, group targeting, scanning, and per-skill usage and cost in analytics. ChatGPT has workspace skills with owners and usage. Salesforce, Decagon, Sierra, Glean and Rovo version and approve well inside their own runtimes. So “no governance” is no longer true inside one tool. What breaks is between tools:

Seam	What breaks
Across AI tools	One skill in Claude, ChatGPT/Codex, Copilot and Cursor; each admin console sees only its copy.
Apps → assistants	Well-governed in Gong, Harvey or Decagon; copied into Claude it becomes an ungoverned fork.
Skills ↔ data	A skill encodes a metric, price or policy; the source changes and nothing links the two.
Measurement	Usage per vendor; nobody can say “this skill ran 4,000 times across 3 tools, with this outcome”.
Security	Scanners flag risky skills but nothing approves a version, pins it and revokes it everywhere.
Migration	Custom GPTs retire 11 Dec 2026 (some Enterprise workspaces 11 Feb 2027); sharing resets and custom actions do not migrate.

This also answers the brief’s “who should not be targeted yet”: organisations standardised on one AI tool (its own console is good enough) and CX teams running Decagon or Sierra (the agent is the product and is governed in place). The qualifying question for every lead: *“How many AI tools are officially allowed, across how many functions?”*

Correction 6 — The pain feeler has a real title

Sweeping personas across tech and non-tech showed who feels the seams most, who pays, and who merely creates the sprawl. The strongest finding: companies are now creating a role whose job is exactly this. GitLab’s public handbook defines an *AI Transformation Owner* per function who “maintains the canonical skills, prompts, and agents used by the function” and reports value to the exec sponsor; LinkedIn is hiring “AI Builder, GTM Enablement” roles to “consolidate one-off solutions into durable systems” and “establish standards for lifecycle management... and governance”. The owner of canonical process is the pain feeler; the title is searchable, which makes it a target list.

Persona	Pain	What they use today	Where it breaks	Role
Head of AI / AI CoE	High	Claude and ChatGPT admin analytics, Agent 365, Larridin	No cross-vendor, per-skill view	Buyer
Owner of canonical process (ATO, GTM Enablement)	High	Role-based plugins, Glean, Gong / Salesforce agents	Same process in several tools; value reporting split	Pain feeler
Platform / DevEx lead	High if multi-tool	Vendor admin planes, sx, Tessel	Across AI tools; release control	Champion
CDO / data governance	Rising	Atlan, Databricks UC skills (Beta), dbt	Skills ↔ data	Buyer in Atlan accounts
AppSec	Medium-high	Snyk agent-scan, Cisco skill-scanner, Zenity	Findings not tied to release/revoke	Blocker → ally
Engineer / tech lead	Low-medium	In-repo CLAUDE.md, AGENTS.md, git	Drift across repos	Wedge
Seller / marketer	Low	Their own prompts and GPTs	They create the sprawl	Consumer
CX ops on Decagon / Sierra	Low	In-app versioning	Solved in place	Not yet

Part III — The landscape, and where Atlan wins

Every major platform added skill or agent governance in 2026, and each one stops at its own edge.

Layer	Who (status, Sept 2026)	Gap Atlan can own
AI tool admin planes	Claude org skills + analytics; ChatGPT workspace skills; Copilot org instructions (GA Apr 2026); Cursor team rules	One version and one usage view across all of them

Layer	Who (status, Sept 2026)	Gap Atlan can own
Cloud agent registries	AWS Agent Registry (GA 31 Aug 2026); Google skill registry (preview); Microsoft Agent 365 (GA 1 May 2026); Databricks UC skills (Beta)	They govern their own cloud; none reaches laptops or other assistants
Skill package managers	sx (Sleuth), Tessel, JFrog skills registry	Early; no org ownership, cross-function usage or data lineage
Security scanners	Snyk agent-scan (ToxicSkills: 13.4% of 3,984 public skills critical; 76 malicious payloads), Cisco, Palo Alto, Zenity	Findings not wired to approve / pin / revoke — partner, don't compete
Apps pushing context out	Gong, Highspot, Salesforce, Harvey, Jasper, Typeface via MCP	MCP moves data and brand rules, not owned, versioned skills

Atlan wins in three places. It is **neutral across AI tools**, which no platform vendor will be. It links **skills to the governed data they encode** — glossary, metrics, lineage, certification — so a skill is flagged when its underlying metric changes; no AI-tool vendor can do that. And it gives the Head of AI a **per-skill value view across vendors**, which today means stitching together several dashboards. The threat to watch is Databricks, whose skills are already catalog objects served over MCP, though Beta and Databricks-centred.

Part IV — The channel, and distribution outside and inside the organisation

Outside: breadth

The primary channel is open-source, GitHub-native distribution, chosen because the behaviour match is observed: Anthropic distributes its official skills as a public GitHub repo (175,633 stars, 20,782 forks in about a year, verified via API), four independent awesome-claude-skills lists exist, and the sharpest pain evidence came from someone in this exact audience launching a tool for this exact pain on Show HN. Day 0 is a data-drop (“we scanned the agent-skill ecosystem”), the CLI is the conversion destination, and HN, Twitter, LinkedIn and awesome-list PRs are the compounding tail.

The two strongest alternatives were steel-manned and rejected. Public skill directories (skills.sh) have reach but no team primitive and optimise the individual install that causes drift. Platform partnerships (AWS, Anthropic) validate the category but are competitors or quarter-long BD cycles. One more is worth naming: *Atlan's existing customers*. It is not a scale channel, but it is the fastest route to the first activations, because the vendor is already approved and the CSM already knows the data leader.

Inside: depth, through hubs

Outside channels only ever reach engineers. The spread into sales, finance and marketing has to be designed. Inside an organisation, distribution comes from *hubs*: teams whose knowledge hundreds of colleagues consume — the data team (metric definitions), product marketing (messaging), platform engineering (standards), GTM enablement (seller workflows). A hub adopts for its own reasons: one version, reaching everyone's AI tool, with usage that proves the hub's work matters. Skills depend on each other — the QBR-deck skill uses marketing's messaging, finance reports use the data team's metrics — so once one hub is governed, dependent skills pull the next hub in. That dependency pull is how the registry crosses from tech into non-tech without a separate sale.

Before any admin install

An earlier version of this plan used the GitHub App as the org-wide scan. That was wrong: nobody installs a GitHub App on an org casually, so an install is the *signal* that a champion is already convinced, not the way in. The org-wide picture must come first, using only access people already have:

Step	What happens	Why it needs no admin
1. Local scan	The engineer runs the CLI on their laptop.	Their own machine.
2. Org scan	Code search with the engineer's existing GitHub login across every repo they can read, plus CODEOWNERS and commit authors to map skills to teams.	Read-only, their own permissions, nothing leaves the machine.
3. Duplicate-cluster PRs	One PR per large cluster in the other team's repo: “1 of 4 drifted copies — pin to one?”	Anyone can open a PR on repos they can read; it lands in the review queue.

Step	What happens	Why it needs no admin
4. Claim your skills	Each team with unowned skills gets one ask: add an owner.	Claiming is that team's first act of adoption.
5. Colleague rollup	Opt-in, hashed and redacted manifests merge by email domain; the org view unlocks at 3+ contributors.	The only step that shares anything, and only by consent.
6. Cross-functional sharing	Skills whose output is used outside engineering are shared as links to the skill's page in the Atlan dashboard, over Slack, with permission.	Uses the person's own Slack connection.
7. Champion → App	Whoever keeps reviewing the PRs and asking for “one place” installs the GitHub App.	The install confirms the champion; it is a conversion metric.

Cross-functional sharing, concretely: the skillsdrift-bridge skill

Almost everyone now has Slack connected to Claude or Codex, so the last mile runs through the assistant itself. The skillsdrift-bridge skill reads an existing drift report (skillsdrift output, a registry-import manifest, a markdown report or an HTML card), decides which skills are cross-functional by asking *who uses what this skill produces?*, finds the few teammates who rely on each one, and shares *a link to that skill's page in the organisation's Atlan dashboard* rather than a file. A file is a copy, and copies are how drift starts; a link is the governed version, and opening it means signing in with a work account — so the recipient is onboarded onto Atlan by the act of reading what a colleague sent.

Consent is layered because messages go out under the user's name: the user approves every destination and the exact text; owners are asked before their skill is named to other teams; security-flagged skills are never shared (the owner hears privately, flag type only); people outside the company are held back by default; and a local log stops anyone hearing the same finding twice. “Atlan” wording is used only when a link really points at the organisation's Atlan workspace — where there is none yet, the skill falls back to a card link and says recipients will not be onboarded.

The honest math

Taking the challenge's implied scale literally — 100 activated ÷ 50% deployment ÷ 30% decision-maker conversion ≈ 667 Hop-1 orgs — assumes one team per organisation, and the ~30 people per org was itself worked backwards from the target, so it cannot also validate it. The 30% rate is also optimistic; bottom-up conversion into enterprise decisions usually lands well below that. The hub model changes the unit: one organisation produces several activated teams (data, then finance, sales ops, marketing).

activated teams = organisations activated × teams per organisation

If teams inside one organisation count separately, the goal needs tens of organisations and the inner loop does most of the work; if not, it remains a multi-quarter, multi-wave goal. I would rather ask Atlan than assume — the brief invites exactly that question. Either way the rates are bets with named levers, instrumented so real data overrides them.

Part V — Worked example: running the loop on GitLab

GitLab is an Atlan customer and publishes how it runs AI internally, which makes it possible to run the whole loop on a real organisation from public sources. (Its analytics dbt repository, once public, now requires sign-in, so this uses the handbook only.)

What GitLab has: Claude Enterprise for every team member, plus Glean, Relevance.ai and GitLab Duo — the data team also uses OpenCode alongside Claude Code. An operating model called *Hub & Spoke & Hub*: a central Enterprise AI team under the CIO, one AI Transformation Owner per function, and a Champion community per function. And a doctrine page, *Prompts are Process*, which says every prompt, skill, plugin or agent is process and must be shared, versioned and owned — with the example of a VP of Sales changing the motion once and every AE picking it up. GitLab has independently written Atlan's thesis down as policy.

Where it breaks, in their own words: the doctrine is “*deliberately tool-agnostic*” but the tooling is not — the same process lives in Claude plugins, Glean skills, Relevance agents, the Duo AI Catalog and repo skills, with governance mapped per platform; “consume before build” has no catalog across tools; skills shared across repos “have to be symlinked into your local skills directory”; promoting a repo skill to function scope means copying it;

and ATOs must report value while usage sits in four dashboards.

Seat	At GitLab
Buyer	Director, Enterprise AI (Enterprise Technology & AI, under the CIO)
Pain feelers	AI Transformation Owners; Enterprise AI engineers
Hubs	Sales ATO (owns the canonical call-prep skill); data team (shared skills across four repos)
Connectors	Champions, #enterprise-ai-collab, role-based Claude plugins
Pitch	"Prompts are Process — with the tooling to match": publish once to every platform, promote between scopes without copying, one per-skill value report for the ATOs
Risk	GitLab dogfoods its own Duo AI Catalog, which covers agents, flows and MCP inside GitLab but not Claude or Glean skills — position Atlan across platforms, with Duo as one destination

Three questions to take to GitLab through Atlan's existing relationship: where does an ATO check whether something already exists today, across all five platforms? When Sales shifts its motion, how many places does call prep change? How does an ATO report value per skill?

Part VI — What was built, and what is blocked

The constraint throughout: it all works without the Atlan product existing. The stack ends at the waitlist; card URLs stand in for Atlan share links; the import manifest ([atlan-registry-import/1](#)) is the portable demand object that makes later conversion a mechanical import. Status is from the build logs; the server was not re-verified in this pass, because it was not reachable from my working environment.

Build	Status	Blocker
skillsdrift CLI — scan, drift diffs, 11 security heuristics, three-layer report, manifest, check-in mode	Built and tested	npm publish (the name is currently unclaimed)
Scanner service + State of Skill Drift index + weekly delta	Built	Domain serving
Card generator + gallery (≤50 KB, redacted, no scores)	Built	Domain serving
X oracle bot (@skillsdrift), human-approval gate enforced in code	Built, draft-only	X API access and credentials
GitHub App service — HMAC webhook, weekly cadence, blast-radius PRs, no auto-fix	Built	App registration needs a public webhook URL
skillsdrift-bridge skill — report triage, cross-functional selection, Slack sharing of Atlan links	Built and dry-run tested	A real report sample to tune parsing; an Atlan workspace for real links
Waitlist backend (title + domain capture)	Not built	Blocks all measurement
Org scan (own GitHub access), duplicate-cluster PRs, claim-your-skills	Designed	None — code only
Colleague rollout (opt-in, hashed, by domain)	Designed	Needs the backend on the domain
Metric-skill kit (dbt definitions → skills, stale-skill check on PRs)	Designed	None — code only

The bottlenecks, in order: (1) drift.aryaman.tech resolves to the VPS but does not serve pages yet — a reverse proxy and TLS certificate unblock the cards, index, waitlist and GitHub App webhook at once; (2) the code needs to be reachable (a GitHub push) so the remaining builds can be completed and audited; (3) the account steps only I can do — npm publish, App registration, X keys; (4) no demand data yet, which is what the brief values most; (5) Atlan-dashboard sharing cannot be tested for real without an Atlan workspace — worth asking for a sandbox.

The original builds passed an adversarial verification round, where a separate AI pass acted as auditor and caught three misses the builder's tests had passed (a too-narrow skills-directory detector, drafts shipping pre-approved, stub cards). The bridge skill went through two dry runs that caught more than a dozen issues, from suggested owners being shown as confirmed to templates contradicting the wording rules.

Part VII — The campaign, the gates, and the next experiments

Audience: engineers at multi-tool organisations with skill sprawl, self-selected by where they discover tools; decision-makers reached by the public data-drop. **Offer:** a free 30-second audit where nothing leaves the machine. **Assets:** the repo, the data-drop, the cards, the weekly delta, the bridge skill. **Destination:** scan →

three-layer report → org scan → manifest → waitlist. **CTAs:** run the scan and share the card; then import this inventory into a governed pilot. **Launch:** Day 0 data-drop on HN and LinkedIn; Days 1–7 give-first cards to the people whose work founded the thesis; Days 7–30 awesome-list PRs gated on real usage; a timed campaign around the custom-GPT retirement for multi-tool organisations.

Stage	Continue if	Change if	Stop if
Outside → first scan	Orgs with lead-visible evidence each wave	Scans cluster solo → add an invite-a-teammate ask	Two waves underperform
Local → org scan	A healthy share of local scans go on to an org scan	Low → the local report isn't making the case to look wider	—
Org scan → teams engaged	Duplicate PRs merged, skills claimed across ≥3 teams	PRs ignored → send the inventory to the operating-model owner directly	—
Tech → non-tech	Share-link opens from ≥1 non-engineering function	Engineering-only after the window → run the bridge-skill play in one function	—
Champion	GitHub App installed; time from first org scan to install falling	Champions engage but DMs don't → change the exec framing	—
Activation	Owners + versions; ≥3 functions; ≥2 AI tools; one owner update reaching every user in every tool	Reviewed quarterly against the conversion bets	—

Next experiments, cheapest first: fix the domain and publish the CLI; ship the waitlist; run five interviews with owners of canonical process (two at Atlan customers); build the org scan; run the full inside loop in one design-partner account; get Atlan's definition of a “team”.

Part VIII — Devil's advocate on the whole thesis

Attack	Answer	Still open?
The platforms will close the gap themselves	Each governs its own tool; the pain is between tools and between skills and data	Watch Databricks and Microsoft
“100% ungoverned” is definitional — public libraries keep ownership in git, not in the file	Agreed; the data-drop must say what it measures and show internal-style drift, not only public repos	Yes — reframe before Day 0
Enterprises block unknown npm packages	Entry is local-only and read-only; existing Atlan accounts skip the trust gap	Partly
Zero telemetry makes the gates unmeasurable	Signals come from opt-in actions: waitlist, rollup, share-link opens, App installs	Needs the backend
Hop 1 carries social risk — nobody volunteers an audit that makes their team look bad	Frame findings as blast radius and next steps, never blame; PRs useful even if Atlan is ignored	Test in interviews
Unsolicited PRs and Slack messages feel like spam	One per cluster, human-approved, under the person's own name; per-destination consent	No
The exec memo overclaims “no security review”	Say “no security review marker found”; a scan cannot see a review	Fix wording
Non-tech crossing is theoretical	Instrumented: count functions using a governed skill; one function = stalled	Yes — no data yet
It redefines the goal it was given	The redefinition is a question for Atlan, not an assumption	Ask Atlan
No demand evidence	True. It is the first thing to fix	Yes

Part IX — How the work happened (the AI-native answer)

The loop: an orchestrator agent on my VPS plans, delegates and verifies; Claude Code executes written briefs with mechanically verifiable done-criteria; every round is verified by artifact — tests run, real scans executed, greps checked — never by the executor's self-report. This final pass used Claude as analyst and builder: parallel research agents for the landscape, a devil's-advocate review of the whole document, a persona sweep, a worked example read directly from GitLab's handbook, and a skill built and dry-run tested with stubbed Slack tools.

Where the AI was wrong, concretely:

- The original research misattributed the sharpest drift quote to the wrong HN commenter (caught by API re-verification).
- An early strategy section fabricated a plausible “four parallel research subagents” narrative (caught by cross-reading).
- The orchestrator proposed free fragments — score badges, a standalone MCP endpoint, a marketplace land-grab — that would have cannibalised the registry; it took two human corrections before the anti-fragmentation gate was written down.
- The adversarial audit caught drafts shipping pre-approved, which the builder's tests had passed.
- In this pass, research agents disagreed on whether Claude's org skills have version history; checking the source directly showed they do, which changed the pain statement.
- I first cited GitLab's dbt project as public; the API returned 404 without sign-in, so the worked example was rebuilt on the handbook.
- The AI first placed the GitHub App as the way into an org and first built the sharing skill as a scanner; both were corrected by me (the install is a champion signal; the skill should consume the drift report and share links).

The decisions that were human:

- Betting the motion on organic, community-led distribution over faster-but-unexecutable alternatives.
- The honesty posture as non-negotiable: no fabricated numbers, disclosure everywhere, zero telemetry by design, and never presenting my own pages as Atlan's.
- Every correction in Part II, and the inside-the-org focus: org scans before the App, hubs over individuals, links through the Atlan dashboard so that sharing is onboarding.

That is the operating thesis the exercise was testing: AI multiplies execution; a human decides what the execution is for.

Open questions for Atlan

- Does “100 activated teams” count several teams inside one organisation separately?
- Could I have a sandbox Atlan workspace, and what is the import path and URL scheme for a skill's page, so shared links onboard people for real?
- Which existing customers have an AI operating-model owner (an ATO, a Head of AI) who would be a design partner?
- Is the strategy memo available? It was the one input I could not access.

Sources

— Atlan: challenge brief (GTM Candidate Challenge 2); co-founder talk, Agent Registry demo and Software Factory demo (recovered transcripts); atlan.com (Context Engineering Studio, “Agent Skills”).

— Anthropic: Provision and manage skills for your organization — support.claude.com/en/articles/13119606; Giving admins more visibility and control over Claude usage and spend (2 Jul 2026) — claude.com/blog.

— OpenAI: Skills in ChatGPT — help.openai.com/en/articles/20001066; Custom GPT retirement and migration FAQ — help.openai.com/en/articles/20001519.

— GitHub: Copilot organization custom instructions GA (2 Apr 2026) — github.blog/changelog.

— AWS Agent Registry GA (31 Aug 2026) — aws.amazon.com/about-aws/whats-new; Google Gemini Enterprise skill registry — docs.cloud.google.com; Microsoft Agent 365 GA (1 May 2026) — microsoft.com/security/blog; Databricks Unity Catalog skills (Beta) — docs.databricks.com/aws/en/agents/uc-skills.

— Snyk: ToxicSkills study and agent-scan — snyk.io/blog, github.com/snyk/agent-scan; sx by Sleuth — github.com/sleuth-io/sx; Tessl skills — tessl.io/blog.

— GitLab handbook: Enterprise AI; Hub & Spoke & Hub; AI Transformation Owner; Prompts are Process; AI tools (Claude, Glean, Relevance.ai); Data team AI Agent Setup and Agentic Tool Development — handbook.gitlab.com. GitLab blog: How GitLab fosters AI-fluent teams (2 Sep 2026); AI Catalog (14 Jan 2026).

— LinkedIn job postings: Manager, AI Builder GTM Enablement; AI Builder, GTM Enablement (Sept 2026).

— dbt community size (100K+) — getdbt.com/community.

— My own scans: 951 public skills across anthropics/skills, vercel-labs/skills, microsoft/azure-skills and ComposioHQ's list; zero owner fields, zero version markers (see Part VIII on what this does and does not measure).